Ankit Raj
6411390
Sogeti/Capgemini

🧠 LangGraph Quiz

📌 Section A: MCQs (1 mark each)

1. What is LangGraph primarily used for?
a) Data visualization
b) Building graph-based AI workflows
c) Database management
d) Web scraping

---

2. In LangGraph, nodes represent:
a) Database tables
b) Functions or steps in a workflow
c) UI components
d) API keys

---

3. What is the main advantage of graph-based workflows?
a) Linear execution only
b) No need for state
c) Flexible branching and looping
d) Faster internet speed

---

4. What does “state” represent in LangGraph?
a) Server location
b) Memory shared across nodes
c) Programming language
d) UI design

---

5. Which component decides the next step in a LangGraph workflow?
a) Tokenizer
b) Edge

c) Prompt
d) Dataset

---

6. A multi-step agent is best described as:
a) A single API call
b) A system executing multiple connected steps
c) A database query
d) A static webpage

---

7. What is the purpose of conditional edges in LangGraph?
a) To delete nodes
b) To control flow based on conditions
c) To store data
d) To train models

---

8. Which of the following is required to define a graph in LangGraph?
a) Nodes and edges
b) Only APIs
c) Only datasets
d) Only UI components

---

9. StateGraph in LangGraph is used for:
a) Styling UI
b) Managing workflow state and transitions
c) Creating databases
d) Sending emails

---

10. What is the output of a node in LangGraph?
a) Only text
b) Updated state
c) Images only
d) HTML code

📌 Section B: Short Answer (2–3 marks each)

11. Explain the concept of graph-based workflows in LangGraph.

Ans: **Graph-based workflows represent processes as nodes (steps) and edges (connections)They allow non-linear execution, including branching, looping, and conditional logic, making workflows flexible and dynamic.**

---

12. What is state management and why is it important in multi-step agents?

Ans: **State management refers to maintaining shared data across all steps of a workflow. It is important because:**

- **It enables communication between nodes**
- **Helps track progress**
- **Supports multi-step reasoning**

---

13. Differentiate between nodes and edges in LangGraph.

Ans:

Nodes	**Edges**
Represent actions/functions	**Represent connections**
Perform operations	**Decide flow direction**
Modify state	**Route between nodes**

---

14. What is a StateGraph and how does it help in building agents?

Ans: **StateGraph is a structure in LangGraph used to:**

- **Manage state updates**
- **Define workflow steps (nodes)**
- **Control transitions (edges)**

It helps build structured AI agents with memory and flow control

---

15. Give one real-world example where a multi-step agent can be useful.

Ans: **A customer support chatbot that:**

- **Receives query**
- **Classifies intent**
- **Retrieves data**
- **Generates response**

📌 Section C: Coding-Based Questions (4–5 marks each)

16. Consider the following code:

```
def step1(state):
    return {"msg": "Hello " + state["name"]}
```

👉 What is happening in this function? Explain.

Ans: □ Takes input from **state["name"]**

□ Concatenates it with "Hello "

□ Returns updated state with "msg"

---

17. Write a simple LangGraph workflow with two steps:

- Step 1: Take user input
- Step 2: Generate a response

(You can write pseudocode)

Ans: state = {"input": "", "response": ""}

```
def step1(state):

    state["input"] = user_input()

    return state

def step2(state):

    state["response"] = generate_response(state["input"])

    return state

# Graph

start → step1 → step2 → end
```

---

18. How would you add a conditional step in a LangGraph workflow? Explain with logic.

Ans: Logic:

```
def condition(state):

    if "exam" in state["input"]:

        return "exam_node"

    else:

        return "assignment_node"
```

Flow:

Input → Condition → (Exam Node / Assignment Node)

📌 Section D: Case Study (5–8 marks)

19. Design a multi-step agent using LangGraph for a “Student Assistant Bot” that:

- Takes a question
- Checks if it is about assignments or exams
- Routes to the correct handler
- Returns a response

👉 Explain:

- Nodes
- Edges
- State

Ans: Sate

```
state = {

    "question": "",

    "type": "",   # exam or assignment

    "answer": ""

}
```

Node

□ Input **Node** → takes question

□ Classifier **Node** → identifies type

□ Assignment **Node** → handles assignment queries

□ Exam **Node** → handles exam queries

□ Response **Node** → returns answer

Edges:

Input → Classifier

Classifier → Assignment Node (if assignment)

Classifier → Exam Node (if exam)

Assignment/Exam → Response

Flow:

□ User asks question

□ System classifies intent

□ Routes to correct handler

□ Returns answer

📌 Section E: Advanced Conceptual (5 marks)

20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?

Ans:

Linear Pipeline	Graph-Based Workflow
Fixed sequence	Flexible structure
No branching	Supports branching
No loops	Supports loops
Less adaptive	Highly adaptive

□ **Supports complex decision-making**

□ **Enables multi-step reasoning**

□ **Allows dynamic routing**

- **Maintains state across steps**
- **Ideal for AI agents**